40 Days of LeetCode Challenge

40 days, 40 problems
DAY :4

LEETCODE PROBLEM NO. 9: Palindrome Number

Problem Link : https://leetcode.com/problems/palindrome-number/description/
Easy · Number manipulation · arithmetic operators · Mathematical Logic · Digital Reversal.
STEP BY STEP SOLUTION:-

Step 1. Understand the question :-

The objective is to determine whether a number remains the same when its digits are reversed.

For example:

- 121 reversed is 121, so the result is true.
- 123 reversed is 321, so the result is false.
- 1221 reversed is 1221, so the result is true.
- 1234 reversed is 4321, so the result is false.

A straightforward approach would be to reverse the entire number and compare it with the original. However, reversing the entire integer can cause integer overflow for sufficiently large values.

To avoid this issue, we can reverse only the second half of the number and compare it with the first half. This approach also avoids string conversion.

Key idea: A palindrome has matching digits on both sides of its center. Therefore, checking the two halves is sufficient.

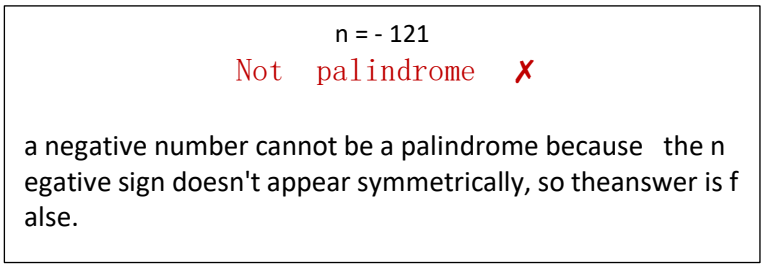
Step 2. Translating it into simpler language with examples :-

Imagine looking at a number in a mirror. If its digits read the same in reverse order, it's a palindrome!
Example 1:
Input: x = 121
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.

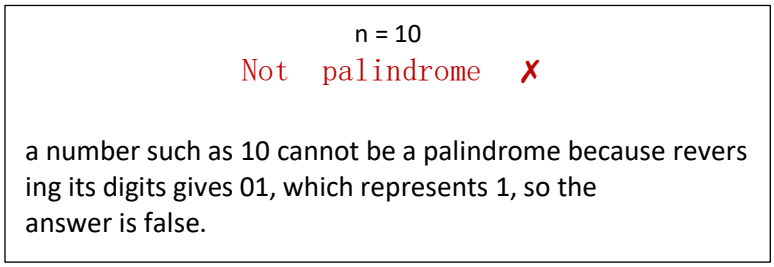
n = 121
Palindrome ✓

121 reversed is 121, so the answer is true.

Example 2:
Input: x = -121
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.



Example 3:
Input: x = 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.



Step 3. Identify the pattern

The problem follows the number manipulation and digit reversal pattern.

We'll use three important operations:

- x % 10 extracts the last digit.
- x / 10 removes the last digit using integer division.
- reversedHalf * 10 + digit appends the extracted digit to the reversed half.

Instead of reversing the entire number, we'll reverse only half of it. This avoids the risk of integer overflow when rev ersing large integers.

A. Modulo operator %

The expression n % 10 returns the last digit of a positive integer.

For example:

- 121 % 10 = 1
- 12 % 10 = 2

- 1 % 10 = 1

B. Integer division /

The expression n / 10 removes the last digit when both operands are integers and n is non-negative.

For example:

- 121 / 10 = 12
- 12 / 10 = 1
- 1 / 10 = 0

Together, these operations allow us to process the digits from right to left without converting the number into a stri ng.

Step 4. Build the logic

Here's the approach we'll follow:

1. If x is negative, return false.
2. If x ends in zero but isn't zero itself, return false.
3. Create reversedHalf = 0.
4. Extract digits from the end of x and append them to reversedHalf.
5. Stop when reversedHalf becomes greater than or equal to the remaining x.
6. Compare both halves:
    - For an even number of digits, check x == reversedHalf.
    - For an odd number of digits, check x == reversedHalf / 10, ignoring the middle digit.

Algorithm

1. Start.
2. Read integer x.
3. Return false if x < 0 or if x % 10 == 0 and x != 0.
4. Initialize reversedHalf = 0.
5. While x > reversedHalf:
    - Extract digit = x % 10.
    - Update reversedHalf = reversedHalf * 10 + digit.
    - Update x = x / 10.
6. Return true if x == reversedHalf or x == reversedHalf / 10.
7. Otherwise, return false.
8. Stop.

Pseudocode

ALGORITHM IsPalindrome(x)

```
BEGIN
  IF x < 0 OR (x MOD 10 = 0 AND x != 0) THEN
    RETURN false
  END IF

   reversedHalf ← 0


  WHILE x > reversedHalf DO
    digit ← x MOD 10
    reversedHalf ← reversedHalf * 10 + digit
    x ← x DIV 10
  END WHILE

  RETURN (x = reversedHalf) OR
       (x = reversedHalf DIV 10)
END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 9.

```
class Solution {

  public boolean isPalindrome(int x) {

    if (x < 0 || (x % 10 == 0 && x != 0)) {

      return false;

    }

    int reversedHalf = 0;

    while (x > reversedHalf) {

      reversedHalf = reversedHalf * 10 + x % 10;

      x /= 10;

    }    return x == reversedHalf || x == reversedHalf / 10;

  }

}
```

How to submit

1. Open LeetCode Problem 9. Palindrome Number
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

Line 1 — Method declaration

public boolean isPalindrome(int x)

The method accepts an integer and returns a Boolean value: true or false.

Line 2 — Handle special cases

if (x < 0 || (x % 10 == 0 && x != 0))

This checks whether the number is negative or ends with zero without being zero itself. Either condition means the number cannot be a palindrome.

Line 3 — Initialize the reversed half

int reversedHalf = 0;

This variable stores the digits we extract from the right side of the number, in reverse order.

Line 4 — Process half the digits

while (x > reversedHalf)

We continue extracting digits until the reversed half catches up with the remaining half.

Line 5 — Build the reversed half & Extract the last digit

*reversedHalf = reversedHalf * 10 + x % 10;*

For example, 1221 % 10 = 1.

If reversedHalf is 12 and the next digit is 2, the new value becomes 122.

Line 6 — Remove the last digit

x /= 10;

For example, 1221 / 10 = 122 because Java integer division discards the decimal part.

Line 7 — Compare the halves

return x == reversedHalf || x == reversedHalf / 10;

The first condition handles even-length numbers. The second handles odd-length numbers by removing the middle digit from reversedHalf.

Dry run: x = 1221

Iteration x before Digit reversedHalf after x after

1	1221	1	1	122
2	122	2	12	12

The loop stops because 12 > 12 is false.

Now compare:

x == reversedHalf

12 == 12 → true

Output: true

For an odd-length example, 12321, the final values become x = 12 and reversedHalf = 123. Since 12 == 123 / 10, the method also returns true.

6. Analyze complexity and optimization

Time complexity: (O(log10 x))

A number has approximately (O(log10 x)) digits. We process only half the digits, but the asymptotic complexity remains (O(log x)).

Auxiliary space complexity: \(O(1)\)

We use a fixed number of integer variables and no additional data structures.

Optimization achieved: Reversing only half the digits avoids constructing the complete reversed integer and its potential overflow. It also avoids string conversion.

SCREENSHOT:-

